

EXPLICACIÓN LÍNEA A LÍNEA

Firmware main.c — ESP32 + INMP441 · Decibelímetro ISO 8253-1

Bioingeniería UNER 2026 · DSP Embebido + FreeRTOS

■ ESP32 Dual-Core	■ INMP441 I2S	■ FreeRTOS	■ UART RS-232	■ ISO 8253-1	■ Filtros Biquad
-------------------	---------------	------------	---------------	--------------	------------------

1. ¿Por qué esta arquitectura dual-core?

El ESP32 tiene **dos núcleos de procesamiento idénticos (Xtensa LX6 a 240 MHz)**. La decisión de separar captura y procesamiento en núcleos distintos no es un capricho: es una necesidad técnica impuesta por los tiempos reales del audio.

El problema que resuelve:

Si la captura I2S y el procesamiento DSP corrieran en el mismo núcleo, el procesamiento (que tarda milisegundos) podría retrasar la lectura del DMA y hacer que el buffer de audio se desborde, perdiendo muestras. Esto se llama **buffer overrun** y produce ruido, silencios o valores erróneos.

	CORE 0	CORE 1
Tarea	task_i2s (prio 10)	task_dsp (prio 9)
Función	Captura audio del INMP441 por bus I2S + DMA	Filtra señal, calcula dB SPL, envía por UART
Prioridad	10 (la más alta)	9 (ligeramente menor)
¿Por qué?	Nunca puede perder una muestra de audio	Puede esperar si no hay datos disponibles
Comunica- ción	Produce: xQueueSend()	Consume: xQueueReceive()
Stack	4096 bytes (solo DMA)	8192 bytes (filtros + math)

Diagrama de flujo entre núcleos:

```
CORE 0: task_i2s INMP441 ■ I2S ■ DMA buffer (int32) ↓ convierte int32 → float32
s_audio_pool[idx] ■ xQueueSend() ■ s_audio_queue CORE 1: task_dsp s_audio_queue
■ xQueueReceive() ■ blk (float*) ↓ 7 filtros biquad en paralelo sum_sq[0..6] ■ RMS ■
dB[0..6] ↓ cada ~200ms uart_send_result() ■ RS-232 ■ Servidor Python
```

■ La cola s_audio_queue tiene QUEUE_DEPTH slots. Si el DSP no consume a tiempo y la cola se llena, xQueueSend() con timeout 0 descarta el bloque sin bloquear la captura. Es preferible perder un bloque ocasional de análisis que perder muestras de audio del ADC.

2. Includes y dependencias

Include	Qué aporta
---------	------------

freertos/FreeRTOS.h	Sistema operativo de tiempo real: scheduler, tasks, semáforos
freertos/task.h	xTaskCreatePinnedToCore(), vTaskDelay(), prioridades
freertos/semphr.h	SemaphoreHandle_t, xSemaphoreGive/Take
freertos/queue.h	QueueHandle_t, xQueueSend/Receive — comunicación entre tareas
driver/i2s.h	API del bus I2S del ESP32 para leer el INMP441
driver/uart.h	API de UART para comunicación RS-232 con Python
driver/gpio.h	Control de pines GPIO (LEDs de estado, etc.)
esp_log.h	ESP_LOGI/LOGW/LOGE — mensajes de debug por serial
esp_timer.h	Temporizador de alta resolución (microsegundos)
nvs_flash.h	Non-Volatile Storage: guarda el factor de calibración en flash
app_config.h	Constantes del proyecto: SAMPLE_RATE, N_BANDS, pines, etc.
biquad_filters.h	7 filtros IIR Biquad pasabanda por banda de octava
db_calculator.h	Función compute_db_spl(), calibrate(), g_cal_factor
uart_comm.h	Protocolo JSON por RS-232: uart_send_result(), uart_rcv_cmd()

3. Buffers y variables globales

```
static int32_t s_dma_raw[DMA_BUF_LEN]; static float s_audio_pool[QUEUE_DEPTH][DMA_BUF_LEN];
static int s_pool_idx = 0;
```

Variable	Tipo	Tamaño	Propósito
s_dma_raw	int32_t[]	DMA_BUF_LEN × 4 bytes	Buffer temporal donde el DMA deposita las muestras crudas del INMP441. El
s_audio_pool	float[][]	QUEUE_DEPTH × DMA_BUF_LEN bytes	Pool de buffers de constantes. Evita malloc() dinámico en tiempo real. La tarea I2S
s_pool_idx	int	4 bytes	Índice circular del pool. Siempre avanza: s_pool_idx % QUEUE_DEPTH da el

¿Por qué pool en lugar de un solo buffer?

Si hubiera un solo buffer compartido, mientras la tarea DSP lo lee, la tarea I2S podría estar escribiendo encima. El pool de QUEUE_DEPTH buffers garantiza que cada bloque que la DSP está procesando no puede ser sobrescrito hasta que no sea consumido.

Variables de la FSM (Finite State Machine):

```
typedef enum { STATE_IDLE, STATE_CALIBRATING, STATE_MEASURING } AppState; static volatile
AppState s_state = STATE_IDLE; static volatile float s_avg_sec = 30.0f; static volatile bool
s_cal_ok = false;
```

Variable	Tipo	Propósito
s_state	enum volatile	Estado actual del sistema. volatile = el compilador no optimiza el acceso porque puede cambiar desde o
s_avg_sec	float volatile	Duración de la ventana de promedio en segundos. También se reutiliza para almacenar ref_db durante
s_cal_ok	bool volatile	true si se realizó al menos una calibración exitosa. Se informa al servidor Python en cada mensaje de e

■ ■ La palabra clave *volatile* es obligatoria para variables compartidas entre tareas en sistemas embebidos. Sin ella, el compilador puede optimizar el acceso y cachear el valor en un registro, haciendo que una tarea no vea los cambios realizados por la otra.

4. task_i2s() — Captura de audio (Core 0)

Esta tarea es la más simple pero la más crítica. Su única responsabilidad es leer el bus I2S y depositar los datos en la cola lo más rápido posible. No hace ningún cálculo — solo captura y transfiere.

Código completo con explicación:

```
static void task_i2s(void *pv) { size_t bytes_read; while (1) { // Loop infinito: corre para siempre // Lee DMA_BUF_LEN muestras del INMP441 // portMAX_DELAY = espera indefinida hasta que haya datos // El DMA llena el buffer automáticamente: la CPU no toca el hardware i2s_read(I2S_PORT, s_dma_raw, DMA_BUF_LEN * sizeof(int32_t), &bytes_read, portMAX_DELAY); int n = (int)(bytes_read / sizeof(int32_t)); // muestras reales // Siguiendo buffer del pool (round-robin) float *dst = s_audio_pool[s_pool_idx % QUEUE_DEPTH]; s_pool_idx++; // Conversion int32 → float32 normalizado [-1.0, +1.0] // El INMP441 pone el dato de 24 bits en bits [31:8] // Desplazar 8 a la derecha extrae los 24 bits significativos // Dividir por 2^23 = 8388608 normaliza al rango [-1, +1] for (int i = 0; i < n; i++) { dst[i] = (float)(s_dma_raw[i] >> 8) / 8388608.0f; } // Envía PUNTERO (no copia) a la cola // timeout 0: si la cola esta llena, descarta el bloque // Preferimos perder un bloque de analisis antes que // bloquear la captura y perder muestras del ADC xQueueSend(s_audio_queue, &dst, 0); } }
```

La conversión int32 → float en detalle:

El INMP441 es un micrófono de 24 bits pero el bus I2S del ESP32 trabaja con tramas de 32 bits. El dato real ocupa los bits [31:8]:

```
Dato crudo (32 bits): [31....24][23....16][15.....8][7.....0] [ dato ][ dato ][ dato ][ 000 ]
MSB LSB Paso 1 – desplazamiento: s_dma_raw[i] >> 8 Descarta los 8 bits bajos (siempre 0)
Resultado: entero de 24 bits con signo (rango: -8388608 a +8388607) Paso 2 – normalización: /
8388608.0f (= 2^23) Divide por el valor maximo posible Resultado: float en rango [-1.0, +1.0]
Ejemplo: Silencio: s_dma_raw = 0x00000000 → float = 0.000000 Maximo +: s_dma_raw = 0x7FFFFFF0
→ float = +0.999999 Maximo -: s_dma_raw = 0x80000100 → float = -1.000000
```

5. task_dsp() — Procesamiento DSP (Core 1)

Esta es la tarea central del sistema. Implementa una **Máquina de Estados Finita (FSM)** con tres estados: IDLE, CALIBRATING y MEASURING. Cada estado procesa los bloques de audio de manera diferente.

Variables locales de la tarea:

```
float *blk; // Puntero al bloque de audio recibido double sum_sq[N_BANDS]; // Acumulador de
energia por banda (double para precision) long n_accum; // Total de muestras acumuladas double
cal_sum_sq; // Acumulador de energia para calibracion (1 kHz) long cal_n; // Muestras
acumuladas durante calibracion int buf_count; // Contador de bloques en la ventana de medicion
```

¿Por qué sum_sq es double y no float?

El RMS requiere sumar el cuadrado de millones de muestras. Con float de 32 bits (precisión ~7 dígitos), la acumulación de errores de redondeo en sumas largas puede producir errores de varios dB. Con double de 64 bits (precisión ~15 dígitos) el error es despreciable incluso acumulando 30 segundos de muestras (30 × 48000 = 1.440.000 valores).

La FSM — tres estados:

Estado	Cuándo activa	Qué hace con el audio
STATE_IDLE	Al arrancar o CMD_STOP	Recibe bloques pero los descarta. Solo atiende comandos UART.
STATE_CALIBRATING	CMD_CALIBRATE	Filtra solo la banda de 1 kHz, acumula 5 segundos de energía, calcula el factor de calibración
STATE_MEASURING	CMD_START	Aplica los 7 filtros Biquad, acumula energía, cada ~200ms calcula dB SPL y envía JSON p

El pipeline DSP en STATE_MEASURING (el más importante):

```
else if (s_state == STATE_MEASURING) { // Para cada muestra del bloque: for (int i = 0; i <
DMA_BUF_LEN; i++) { float x = blk[i]; // Muestra de audio normalizada // Aplicar los 7 filtros
biquad en PARALELO // Cada filtro recibe la misma muestra x // y produce su salida y filtrada
en esa banda for (int b = 0; b < N_BANDS; b++) { float y = biquad_process(&octave;_filters[b],
x); sum_sq[b] += (double)(y * y); // Acumular y^2 } } n_accum += DMA_BUF_LEN; // Sumar muestras
procesadas buf_count++; // Contar bloques // Cada MEAS_WINDOW_BUFS bloques (~200 ms) → enviar
resultado if (buf_count >= MEAS_WINDOW_BUFS) { float db[N_BANDS]; for (int b = 0; b < N_BANDS;
b++) { db[b] = compute_db_spl((float)sum_sq[b], (int)n_accum); } MeasurementResult result;
iso_validate(db, &result); // Compara con limites ISO uart_send_result(&result); // Envia
JSON por RS-232 // Reset acumuladores para la siguiente ventana memset(sum_sq, 0,
sizeof(sum_sq)); n_accum = 0; buf_count = 0; } }
```

¿Por qué filtros en paralelo (no en serie)?

Cada banda de octava necesita medir la energía acústica que HAY en esa frecuencia en la señal original. Si los filtros fueran en serie, cada filtro recibiría la señal ya modificada por el anterior, distorsionando las mediciones. En paralelo, todos reciben la misma señal cruda x y producen mediciones independientes e independientes.

```
PARALELO (correcto): EN SERIE (incorrecto): x F125 y125 x F125 F250 F500 ... x F250 y250 (cada filtro ve la señal x F500 y500 modificada por el
anterior) x F1k y1k ... (todos reciben el mismo x)
```

Cálculo de la ventana de 200ms:

```
// De app_config.h: #define SAMPLE_RATE 48000 // muestras por segundo #define DMA_BUF_LEN 1024
// muestras por bloque DMA #define MEAS_WINDOW_BUFS 10 // bloques por ventana // Tiempo por
bloque = 1024 / 48000 = 21.3 ms // Tiempo por ventana = 10 × 21.3 ms = 213 ms ≈ 200 ms //
Frecuencia de reporte al servidor Python: ~5 veces por segundo
```

6. Manejo de comandos UART — Protocolo con Python

Al final del loop de task_dsp se consulta la cola de comandos UART de forma **no bloqueante** (sin esperar). Esto permite procesar comandos del servidor Python sin interrumpir el pipeline de audio.

```
// uart_rcv_cmd() retorna true solo si habia un comando pendiente // Si no hay comandos,
retorna false y el loop continua inmediatamente Command cmd; if (uart_rcv_cmd(&cmd;)) { switch
(cmd.type) { case CMD_START: // Python dice "iniciar medicion" case CMD_STOP: // Python dice
"detener" case CMD_CALIBRATE: // Python dice "calibrar con X dB de referencia" case CMD_STATUS:
// Python pregunta el estado actual case CMD_RESET: // Python pide reiniciar el ESP32 } }
```

Comando	Parámetro	Qué hace el ESP32
CMD_START	avg_sec (float)	Resetea filtros y acumuladores, cambia a STATE_MEASURING, responde con status JSON.
CMD_STOP	ninguno	Cambia a STATE_IDLE, responde con status JSON.
CMD_CALIBRATE	ref_db (float)	Guarda ref_db en s_avg_sec, resetea filtros, cambia a STATE_CALIBRATING por 5 segundos.
CMD_STATUS	ninguno	Responde inmediatamente con el estado actual y si está calibrado.
CMD_RESET	ninguno	Llama a esp_restart() — reinicio completo del microcontrolador.

7. Inicialización del bus I2S (i2s_init)

El bus **I2S (Inter-IC Sound)** es un protocolo de comunicación digital estándar para audio. Usa 3 cables: reloj de bits (BCK), reloj de word (WS/LRCK) y datos (DATA). El INMP441 actúa como esclavo y entrega datos síncronamente cuando el ESP32 (master) genera los relojes.

```
i2s_config_t cfg = { .mode = I2S_MODE_MASTER | I2S_MODE_RX, // MASTER: el ESP32 genera los
relojes BCK y WS // RX: solo recibe, no transmite (micrófono) .sample_rate = SAMPLE_RATE, //
48.000 Hz // Determina la frecuencia del reloj WS (word select) // BCK = sample_rate × bits ×
```

```
canales = 48000 × 32 × 2 = 3.072 MHz .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT, // El
INMP441 tiene 24 bits utiles pero el protocolo I2S // usa tramas de 32 bits. Los 8 bits bajos
siempre son 0. .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT, // El pin L/R del INMP441 conectado
a GND = canal izquierdo // Solo un micrófono → solo canal izquierdo .use_apll = true, // APLL
= Audio PLL (Phase Locked Loop) // Genera una frecuencia de clock más precisa que el oscilador
// normal del ESP32. Reduce el jitter del clock I2S y mejora // la calidad del audio capturado.
.dma_buf_count = DMA_BUF_COUNT, // Número de buffers DMA .dma_buf_len = DMA_BUF_LEN, //
Muestras por buffer DMA // DMA = Direct Memory Access: el hardware transfiere datos // a RAM
sin intervención de la CPU. La CPU solo se despierta // cuando el buffer está lleno → máxima
eficiencia };
```

Parámetro	Valor	Por qué
mode	MASTER RX	ESP32 genera los clocks, solo recibe (micrófono)
sample_rate	48.000 Hz	Cubre hasta 24 kHz (Nyquist). Cubre todas las bandas hasta 8 kHz con margen.
bits_per_sample	32 bits	INMP441 tiene 24 bits útiles en trama de 32. Los 8 bajos son siempre cero.
channel_format	ONLY_LEFT	Pin L/R del INMP441 conectado a GND = canal izquierdo solamente.
use_apll	true	PLL de audio dedicado: clock más preciso, menos jitter, mejor SNR.
dma_buf_count	DMA_BUF_COUNT (typ 4)	Número de buffers en el pipeline DMA. Más buffers = menos riesgo de underrun.
dma_buf_len	DMA_BUF_LEN (typ 1024)	Muestras por buffer = 1024/48000 = 21.3 ms de audio por interrupción.

8. app_main() — Secuencia de arranque

En ESP-IDF, app_main() es el equivalente de main() en C estándar. Es llamado por el framework después de inicializar el hardware básico. Aquí se hace toda la configuración en orden específico:

```
void app_main(void) { // 1. NVS – memoria flash no volátil // Necesaria para guardar el factor
de calibración entre reinicios esp_err_t nvs_err = nvs_flash_init(); if (nvs_err ==
ESP_ERR_NVS_NO_FREE_PAGES || nvs_err == ESP_ERR_NVS_NEW_VERSION_FOUND) { nvs_flash_erase(); //
Borrar si está corrupta nvs_flash_init(); // Reinicializar limpia } // 2. Inicializar módulos
en orden de dependencias db_calc_init(); // Carga cal_factor desde NVS (o 1.0 si no hay)
biquad_init_all(); // Calcula coeficientes de los 7 filtros uart_comm_init(); // Configura
UART2 a 921600 bps para Python // 3. Crear primitivas de sincronización FreeRTOS s_audio_queue
= xQueueCreate(QueueDepth, sizeof(float *)); // Cola de punteros (no de datos): eficiente, sin
copia de buffers s_data_ready = xSemaphoreCreateBinary(); // 4. Configurar hardware I2S
i2s_init(); // 5. Crear tareas en sus núcleos específicos xTaskCreatePinnedToCore(task_i2s,
"i2s_cap", 4096, NULL, 10, NULL, 0); // nombre↑ stack↑ params↑ prio↑ handle↑ core↑
xTaskCreatePinnedToCore(task_dsp, "dsp", 8192, NULL, 9, NULL, 1); // 6. Informar al servidor
Python que el sistema está listo uart_send_status("idle", g_cal_factor != 1.0f); ESP_LOGI(TAG,
"Sistema listo. Esperando comandos por RS-232."); // app_main() termina aquí – FreeRTOS toma el
control }
```

Paso	Función	Por qué ese orden
1	nvs_flash_init()	Debe ser lo primero: otros módulos dependen de NVS para cargar su configuración persistente.
2	db_calc_init()	Carga cal_factor de NVS. Debe ir antes de crear las tareas que usarán ese factor.
3	biquad_init_all()	Calcula los coeficientes matemáticos de los filtros. Debe estar antes de que task_dsp los use.
4	uart_comm_init()	Abre el puerto serie. Debe estar antes de que cualquier tarea intente enviar mensajes.
5	xQueueCreate()	Crea la cola. Debe existir antes de crear las tareas que la usan.
6	i2s_init()	Activa el hardware I2S. El DMA empieza a capturar inmediatamente.

7	<code>xTaskCreatePinnedToCore()</code>	Las tareas se crean al final, cuando todo lo que necesitan ya está inicializado.
8	<code>uart_send_status()</code>	Notifica a Python que el sistema está operativo y listo para recibir comandos.